Find Closest Value In Bst

AlgoExpert · Easy · Binary Search Trees

The problem

- Return the value in the BST closest to the target (assume one answer).
- Iteratively walk down, pruning half the tree each step.

Intuition

- The BST ordering lets us discard half the tree at every step, just like binary search.

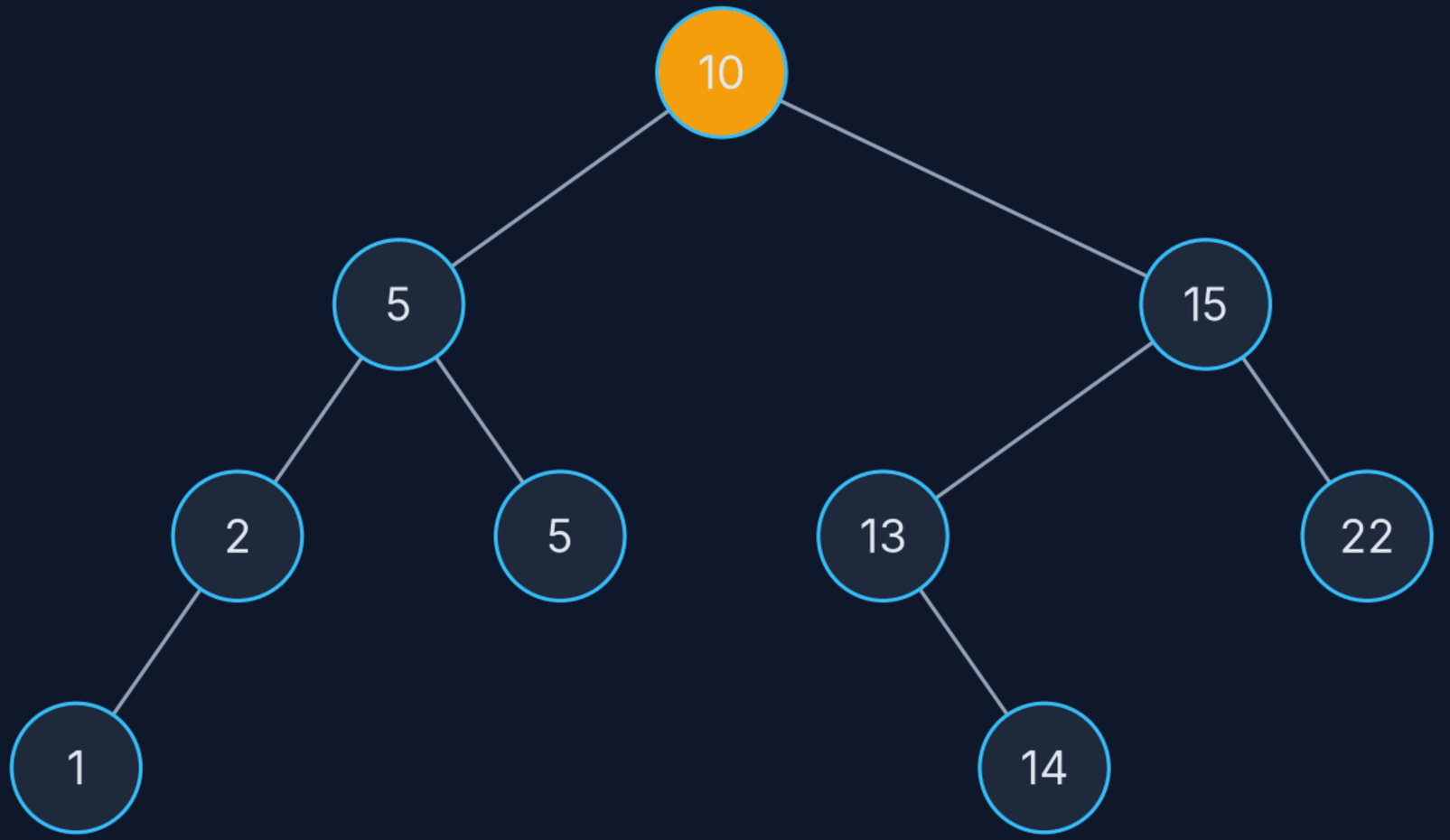
Brute force vs optimal

- Brute force: visit every node → O(n)
- Optimal: follow the BST ordering → O(log n) average
- Worst case O(n) on a skewed tree

Approach

- 1. Start at the root, tracking the closest value
- 2. Update closest if this node is nearer
- 3. Go left if the target is smaller, right if larger
- 4. Stop on an exact match or a null child

Worked example




```
target = 12

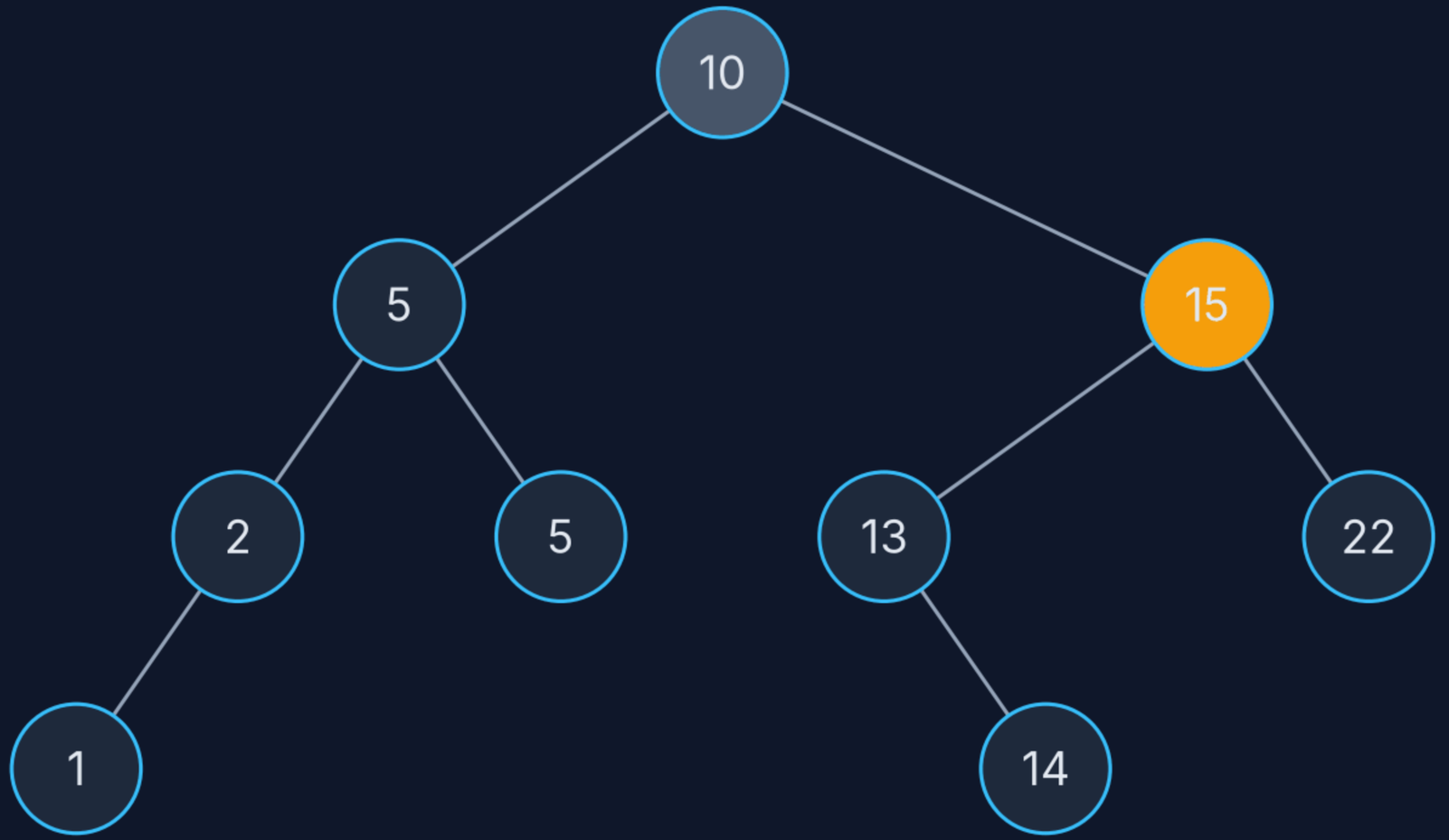
node = 10

closest = 10

go right
```

At 10: closest so far 10; 12 > 10 → go right

Worked example




```
target = 12

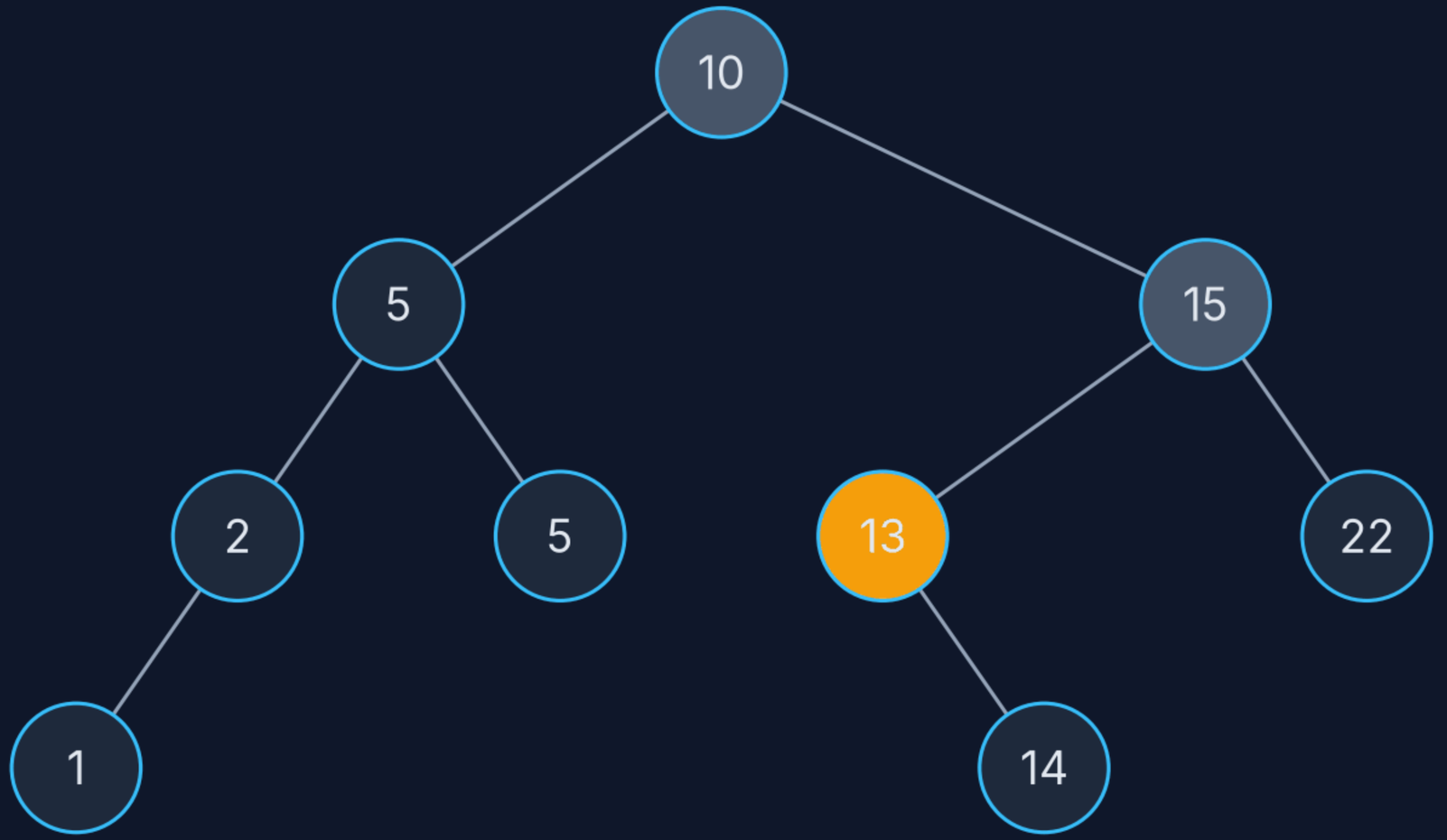
node = 15

closest = 10

go left
```

At 15: closest so far 10; 12 < 15 → go left

Worked example




```
target = 12

node = 13

closest = 13

go left
```

At 13: closest so far 13; 12 < 13 → go left

Java solution

```java
public class FindClosestValueInBst {
    public static int findClosestValueInBst(BST tree, int target) {
        int closest = tree.value;
        BST cur = tree;
        while (cur != null) {
            if (Math.abs(target - cur.value) < Math.abs(target - closest)) closest = cur.value;
            if (target < cur.value) cur = cur.left;
            else if (target > cur.value) cur = cur.right;
            else break;
        }
        return closest;
    }
}
```

Complexity

- Time: O(log n) avg, O(n) worst
- Space: O(1)

Edge cases & pitfalls

- Exact match → stop early
- Skewed (linked-list-like) tree → O(n)
- Use absolute difference; watch integer overflow

Interview tips & follow-ups

- Prefer the iterative version: O(1) space
- Follow-up: k closest values (in-order + window)

Recap

- The BST ordering lets us discard half the tree at every step, just like binary search.
- Complexity: O(log n) avg, O(n) worst